

Lecture 16: Routing — Distance Vector, Bellman-Ford, and BGP

Introduction

A distributed approach to routing, and the protocol that glues the internet together

EC 441 — Introduction to Computer Networking
Boston University

Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

- State the **Bellman-Ford equation** and explain what each term represents
- Trace the **distance-vector algorithm** step by step, showing convergence
- Explain **count-to-infinity** and how split horizon / poisoned reverse help
- Describe **RIP** as a deployed distance-vector protocol
- Explain why one routing algorithm cannot scale to the entire internet
- Define **autonomous system** and distinguish **intra-AS** from **inter-AS** routing
- Describe how **BGP** uses path vectors and business relationships

Key Acronyms

Acronym	Meaning
AS	Autonomous System
ASN	Autonomous System Number
BGP	Border Gateway Protocol
DV	Distance Vector
EGP	Exterior Gateway Protocol
IGP	Interior Gateway Protocol

Acronym	Meaning
IS-IS	Intermediate System to IS
LS	Link State
OSPF	Open Shortest Path First
RIP	Routing Information Protocol
RPKI	Resource Public Key Infrastructure
UDP	User Datagram Protocol

Recall: Two Families of Routing

Link-state (L15):

Every router gets the **complete map**.
Each computes shortest paths
independently.

Analogy:

Having a **full road map** and planning your route.

Distance-vector (today):

Each router knows only its **direct neighbors**
and their distance estimates.

Analogy:

Asking **locals for directions** at each intersection.

The duality (from L15)

Link-state: *local* information distributed *globally* — each router shares its own links with everyone.

Distance-vector: *global* information distributed *locally* — each router shares its distances to all destinations with only its neighbors.

The Bellman-Ford Equation

Define $d_x(y)$ = cost of the least-cost path from node x to node y . Then:

The Bellman-Ford equation

$$d_x(y) = \min_{v \in \text{neighbors}(x)} \{c(x, v) + d_v(y)\}$$

In words: the shortest-path cost from x to y equals the minimum, over all neighbors v , of the direct link cost to v plus v 's shortest-path cost to y .

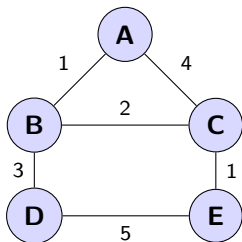
Why it works: optimal substructure

If the shortest path from x to y goes through neighbor v , then the sub-path from v to y must itself be the shortest path from v to y . This is the foundation of **dynamic programming**.

[Wikipedia: Bellman-Ford algorithm]

Apply Bellman-Ford at node A for destination E:

A's neighbors are B and C:



$$d_A(E) = \min \begin{cases} c(A, B) + d_B(E) \\ c(A, C) + d_C(E) \end{cases}$$

Need $d_B(E)$ and $d_C(E)$:

- $d_C(E) = 1$ (direct link)
- $d_B(E) = \min\{1+4, 2+1, 3+5\} = 3$ via C

$$d_A(E) = \min\{1 + 3, 4 + 1\} = \mathbf{4} \text{ via B}$$

Path: $A \rightarrow B \rightarrow C \rightarrow E$, cost $1+2+1 = 4$ ✓

Dijkstra vs. Bellman-Ford

	Dijkstra (L15)	Bellman-Ford (today)
Information needed	Full graph at one place	Only neighbor info
Computation	Centralized	Distributed
Communication	Flood all links to everyone	Exchange DVs with neighbors
Negative weights	Not supported	Supported
Deployed as	OSPF, IS-IS	RIP

The key advantage of Bellman-Ford

No node needs the full topology. Each node only needs:

- 1 Its own direct link costs $c(x, v)$
- 2 Its neighbors' distance vectors $D_v(y)$ for all y

This enables a fully **distributed** algorithm.

The Distance-Vector Algorithm

Each node x maintains a **distance vector**: $D_x = [D_x(y) \text{ for all } y]$

Algorithm

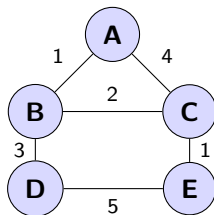
- 1 **Initialize**: $D_x(x) = 0$; $D_x(v) = c(x, v)$ for neighbors; $D_x(y) = \infty$ otherwise
- 2 **Send** D_x to all neighbors
- 3 **On receiving** D_v from neighbor v : for each destination y ,
if $c(x, v) + D_v(y) < D_x(y)$: update $D_x(y)$ and set $\text{next_hop}(y) = v$
- 4 If any $D_x(y)$ changed, **send updated** D_x to all neighbors
- 5 **Repeat** until no changes (convergence)

Key properties:

- **Iterative**: updates continue until convergence
- **Asynchronous**: no global clock needed
- **Distributed**: each node talks only to direct neighbors
- **Self-terminating**: stops when no node has new information

[Wikipedia: Distance-vector routing protocol]

Convergence Trace: Round 0 (Initialization)



Each node knows only direct links:

	A	B	C	D	E
A	0	1	4	∞	∞
B	1	0	2	3	∞
C	4	2	0	∞	1
D	∞	3	∞	0	5
E	∞	∞	1	5	0

Rows = source node's distance vector.

Each node sends its DV to its neighbors.

Convergence Trace: Round 1 (First Exchange)

After receiving and processing neighbors' DVs:

	A	B	C	D	E	Changes
A	0	1	3	4	5	C: $4 \rightarrow 3$ via B; D: $\infty \rightarrow 4$ via B; E: $\infty \rightarrow 5$ via C
B	1	0	2	3	3	E: $\infty \rightarrow 3$ via C
C	3	2	0	5	1	A: $4 \rightarrow 3$ via B; D: $\infty \rightarrow 5$ via B
D	4	3	5	0	5	A: $\infty \rightarrow 4$ via B; C: $\infty \rightarrow 5$ via B
E	5	3	1	5	0	A: $\infty \rightarrow 5$ via C; B: $\infty \rightarrow 3$ via C

Example: Node A updates

$$D_A(C) = \min\{c(A, C), c(A, B) + D_B(C)\} = \min\{4, 1 + 2\} = \mathbf{3} \text{ via B}$$

$$D_A(D) = \min\{\infty, c(A, B) + D_B(D)\} = \min\{\infty, 1 + 3\} = \mathbf{4} \text{ via B}$$

Convergence Trace: Round 2 (Converged)

After second exchange — only one entry improves:

	A	B	C	D	E	Changes
A	0	1	3	4	4	E: $5 \rightarrow 4$ via B (heard $D_B(E) = 3$)
B	1	0	2	3	3	(no change)
C	3	2	0	5	1	(no change)
D	4	3	5	0	5	(no change)
E	4	3	1	5	0	A: $5 \rightarrow 4$ via C (heard $D_C(A) = 3$)

Round 3: All nodes check — no improvements found. **Converged!**

Node A's forwarding table:

Destination	Next hop	Cost
B	B	1
C	B	3
D	B	4
E	B	4

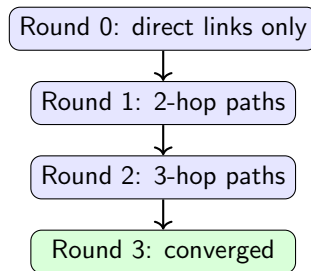
All traffic from A goes via B (A's only low-cost neighbor).

Convergence: How Many Rounds?

General result: DV converges in at most $|V| - 1$ rounds.

Why? The longest shortest path in a graph with $|V|$ nodes has at most $|V| - 1$ edges. Each round of DV extends knowledge by one hop. After $|V| - 1$ rounds, information has propagated across the entire network.

Our 5-node network converged in **2–3 rounds** (typical for small graphs).



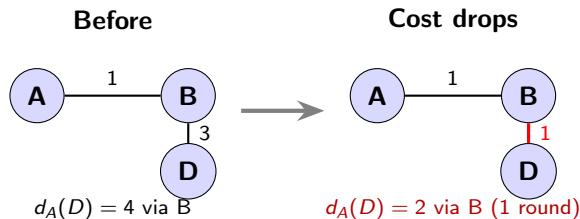
Compare with Dijkstra

Dijkstra needs the **full graph** but runs in one shot: $O(|V|^2)$ or $O(|E| \log |V|)$.

DV needs **multiple rounds** of neighbor exchange but each node only does local work.

Good News Travels Fast

When a link cost **decreases**, improvements propagate quickly:

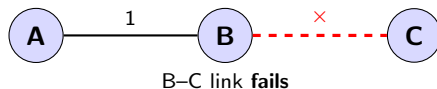


B detects the improvement, updates its DV, tells A. A immediately benefits. Convergence in **one round**.

The asymmetry

Good news travels fast. **Bad news travels slow.**

Bad News Travels Slow: Count-to-Infinity



Before failure: $d_A(C) = 2$ via B, $d_B(C) = 1$ direct.

- ① B detects failure. Checks neighbor A: $d_A(C) = 2$.
B computes: $D_B(C) = 1 + 2 = 3$ via A. **Wrong!** A's route goes through B.
- ② B tells A: $D_B(C) = 3$. A updates: $D_A(C) = 1 + 3 = 4$.
- ③ A tells B. B updates: $D_B(C) = 1 + 4 = 5$.
- ④ Costs increment by 1 each round... **counting to infinity**.

Root cause

B uses information from A **that depends on B itself**. A's route to C goes through B, but B doesn't know that. During convergence, packets for C **bounce between A and B** (routing loop).

[Wikipedia: Count-to-infinity problem]

Fixing Count-to-Infinity

Fix 1: Split Horizon

Rule: If A routes to C via B, then A **does not advertise** its route to C back to B.

Don't tell your neighbor about a route that goes through them.

When B–C fails, B never hears A's stale $d_A(C) = 2$, so B correctly sets $D_B(C) = \infty$.

Fix 2: Poisoned Reverse

Rule: If A routes to C via B, then A **tells B that** $d_A(C) = \infty$.

Actively poison the reverse path — more robust than silence.

B receives $D_A(C) = \infty$ from A, so B won't attempt to route through A to reach C.

Limitation

Split horizon and poisoned reverse fix **two-node loops**. They do **not** prevent count-to-infinity in loops with **three or more nodes**. Larger loops require additional mechanisms (hold-down timers, route poisoning).

[Wikipedia: Split horizon route advertisement]

LS vs. DV: Convergence Summary

	Link-State (OSPF)	Distance-Vector (RIP)
Good news	Fast (one flood + recompute)	Fast (a few rounds)
Bad news	Fast (same mechanism)	Slow (count-to-infinity)
Loop-free?	Yes (full topology)	Not guaranteed during convergence

The defining weakness of distance-vector

Bad news travels slow. After a failure, DV may form routing loops that persist for many update rounds before converging. Link-state protocols avoid this because every router has the complete topology.

This asymmetry is why OSPF has largely replaced RIP in production networks.

RIP: The Simplest Deployed DV Protocol

The **Routing Information Protocol (RIP)** implements the DV algorithm directly:

Parameter	RIP choice
Metric	Hop count (every link costs 1)
Maximum distance	15 hops ($16 = \infty$)
Update interval	Every 30 seconds, full DV broadcast
Transport	UDP port 520
Triggered updates	Immediate update on route change
Timeout	180 seconds without update $\rightarrow$ set routes to ∞

Why hop count?

Simplest metric — ignores bandwidth, delay, congestion. A 2-hop path over 10 Gb/s links = same as 2 hops over 1 Mb/s links.

Why max 15?

Limits count-to-infinity to ≤ 16 rounds (≈ 8 min at 30-sec intervals). Tradeoff: can't use RIP in networks > 15 hops.

[Wikipedia: Routing Information Protocol]

Demo: netstat -rn — The Routing Table

Demo: routing table entries — the output of the routing algorithm

```
$ netstat -rn
Routing tables

Internet:
Destination Gateway Flags Netif Metric
default 192.168.1.1 UGS en0 0
10.0.0.0/24 10.0.0.1 UGS en1 5
127.0.0.1 127.0.0.1 UH lo0 0
192.168.1.0/24 link#4 UC en0 0
```

The **Metric** column is what RIP fills in (hop count). OSPF fills this with computed link-state costs. The routing table structure is protocol-independent.

RIP's place today: dominant in the 1980s–90s. Largely **replaced by OSPF** (faster convergence, richer metrics, no 15-hop limit). Still used in small networks and for teaching.

Why One Algorithm Isn't Enough

Can we run OSPF (or RIP) across the entire internet?

The internet has:

- Over **75,000** independently administered networks
- Hundreds of millions of IP prefixes
- Diverse policies, performance requirements, business relationships

Three barriers:

- 1 **Scale:** OSPF requires every router to store the full topology. With millions of routers, the LSDB and flooding traffic are unsustainable.
- 2 **Administrative autonomy:** Different organizations need different policies. No single metric satisfies a university, a cloud provider, and a mobile carrier.
- 3 **Business relationships:** Routing is driven by economics and contracts. An ISP may refuse to carry a competitor's traffic, even if the path is shorter.

Solution: hierarchical routing

Within an organization: OSPF or RIP (your choice).

Between organizations: a different protocol designed for policy — **BGP**.

Autonomous Systems

An **autonomous system (AS)** = a collection of IP networks and routers under one organization with a common routing policy.

Type	Description	Example	Transit?
Stub	Single upstream connection	Small company	No
Multi-homed	Multiple ISPs, redundancy	University (e.g., BU)	No
Transit	Carries traffic between ASes	Comcast, AT&T	Yes

Each AS has a unique **AS number (ASN)**: originally 16-bit (0–65,535), now 32-bit.

	Intra-AS (IGP)	Inter-AS (BGP)
Scope	Within one AS	Between ASes
Goal	Shortest/cheapest path	Enforce routing policy
Protocols	OSPF, RIP, IS-IS	BGP
Metric	Link cost, hop count	Business relationships
Convergence	Seconds	Minutes

[Wikipedia: Autonomous system (Internet)]

Demo: Identifying ASes on a Path

Demo: traceroute + whois to map a path across ASes

```
$ traceroute -a google.com
1 [AS0] 192.168.1.1 1.2 ms (local)
2 [AS7922] 96.120.56.1 8.4 ms (Comcast)
3 [AS7922] 68.86.103.69 12.1 ms (still Comcast)
4 [AS15169] 108.170.248.1 14.3 ms (Google)
5 [AS15169] 142.250.80.46 13.8 ms (still Google)
```

```
$ whois -h whois.radb.net AS15169
as-name: GOOGLE
descr: Google LLC
```

Traffic crossed from AS 7922 (Comcast) to AS 15169 (Google).

That handoff was determined by **BGP** — the two ASes have a business agreement about which traffic to exchange and where.

What Is BGP?

The **Border Gateway Protocol (BGP)** is the inter-AS routing protocol of the internet. Every AS that connects to the internet runs BGP.

BGP's three jobs:

- 1 **Advertise reachability:** “My AS can reach these IP prefixes”
- 2 **Propagate paths:** “To reach 128.197.0.0/16, go through AS-path [7922, 3356, 111]”
- 3 **Enforce policy:** “I will carry this traffic” or “I refuse”

BGP is often called “the glue that holds the internet together”

Without BGP, the 75,000+ autonomous systems would be isolated islands. Every website you visit, every video you stream, relies on BGP routing your packets across multiple AS boundaries.

[Wikipedia: Border Gateway Protocol]

If a router sees **its own ASN** in the AS-PATH, it **rejects the route** — this prevents loops cleanly, without count-to-infinity.

BGP is neither link-state nor distance-vector. It is a **path vector** protocol.

	Distance Vector	Link State	Path Vector (BGP)
Advertises	Distance to dest	Full local link info	Complete AS-path
Loop prevention	Split horizon	Full topology	Reject own ASN in path
Metric	Numeric cost	Numeric cost	Policy-based

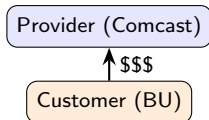
AS-PATH example:

BGP and Business Relationships

Unlike OSPF or RIP, BGP routing is driven by **business relationships**:

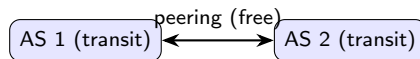
Customer–Provider:

Customer pays provider for internet access.
Provider advertises customer routes globally.



Peer–Peer:

Two ASes carry each other's traffic for free — but only for their own customers, not transit.



The valley-free rule

Traffic flows **up** (customer → provider), across a **peering link** (at most once), then **down** (provider → customer). A well-configured BGP system never routes customer→provider→peer→provider→customer.

[Wikipedia: Peering]

Why BGP Is Hard

- 1 **Policy over optimality:** the “best” route is not the shortest — it’s the one that satisfies business contracts
- 2 **No global view:** unlike OSPF, no single entity sees the full routing table with all policies
- 3 **Slow convergence:** BGP convergence after a failure can take **minutes** (vs. seconds for OSPF)
- 4 **Scale:** the global BGP table contains over **1 million prefixes**
- 5 **Trust-based:** ASes trust their neighbors’ announcements — limited verification infrastructure

BGP incidents

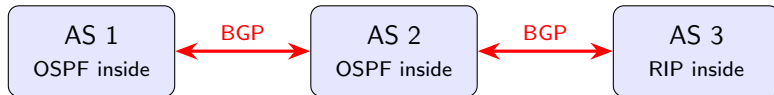
Pakistan/YouTube (2008): Pakistan Telecom announced a more-specific prefix for YouTube. The announcement propagated globally, black-holing YouTube for ~ 2 hours.

Facebook (2021): A config change withdrew BGP routes for all Facebook properties. DNS servers were behind those routes $\rightarrow$ domain names unresolvable. Outage: ~ 6 hours.

[Wikipedia: BGP hijacking]

The Routing Landscape

Lecture	Algorithm	Type	Scope	Deployed as
L15	Dijkstra	Link-state	Intra-AS	OSPF, IS-IS
L16	Bellman-Ford	Distance-vector	Intra-AS	RIP
L16	Path vector	Policy-based	Inter-AS	BGP



Intra-AS: OSPF or RIP (shortest path)

Inter-AS: BGP (policy-based)

LS vs. DV: When to Use Each

Consider. . .	Link-State (OSPF)	Distance-Vector (RIP)
Network size	Large (100s of routers)	Small (<15 hops)
Convergence	Fast	Slow (count-to-infinity)
Memory/CPU	Higher (stores full topology)	Lower (stores only DV)
Complexity	More complex to implement	Simpler
Metric flexibility	Rich (bandwidth, delay)	Typically hop count only

In practice, OSPF has won for most intra-AS deployments. RIP persists in small, simple networks.

Optional further reading on BGP:

- BGP mechanics: eBGP vs. iBGP, route selection (LOCAL_PREF, MED, AS-PATH length)
- Hot potato routing, traffic engineering, RPKI / BGPsec
- Tools: BGP looking glasses (BGP.he.net), RIPE Stat, RouteViews

Today completed the routing algorithm picture:

L13: Forwarding tables and longest-prefix match (data plane)

L14: IP addresses and CIDR prefixes (what goes in the table)

L15: Link-state routing and Dijkstra (control plane, part 1)

L16: Distance-vector, Bellman-Ford, and BGP (control plane, part 2)

Coming up:

- ① **L17 (Tue Mar 31):** IPv4 datagram format, ICMP, NAT, IPv6, DHCP
- ② **L18 (Thu Apr 02):** Wireshark lab — hands-on packet analysis (L13–L17)